

DDTA - Working Metamodel - Functional Requirement

DOCUMENTATION-LAYER CLOSURE CANDIDATE - REVISION 2

This revision supersedes the Revision 1 topology experiment. The two earlier simplified hold-outs that suggested direct MR -> FR parentage are excluded from closure evidence. More realistic access-control and warehouse/order experiments repeatedly exposed hidden responsibility/solution commitments before operational FRs could be written. For the thesis baseline, DDTA therefore adopts a regular MR -> Decision -> FR hierarchy, with a controlled necessity/default Decision rule for the rare singleton-solution case.

1. Definition

A Functional Requirement is a governed, independently assessable **operational obligation** that descends from exactly one Decision and states one coherent behavior, service, transition or observable result that the project/system must provide under the governed context established by its MR and Decision branch.

It must be specific enough to guide realization and verification without prescribing incidental source-code/framework detail.

Fundamental question

Given the parent Decision and an applicable condition/input, which governed subject/capability must perform which functional action on or with which governed object/information, and which observable outcome or failure behavior must follow?

2. Hierarchy and ownership

```
MacroRequirement
|
+-- Decision
|
|   +-- FunctionalRequirement
|   |
|   |   +-- SpecializedRequirement *
```

Closed thesis-baseline rules:

- FunctionalRequirement.parentDecision = exactly 1;
- owning MR is derived from parentDecision -> MacroRequirement and must not become a second topology authority;
- an L2 registry may materialize the MR id as an indexed/mirrored value, but it is derived;
- no Decision is a child of FR;
- FR may have 0..* Specialized Requirements;
- the thesis next formal specialization is SecurityRequirement; Governance/Privacy/Performance remain future/generalization work unless needed by a concrete counterexample.

3. Semantic shape

```
FunctionalRequirement
|- id / lifecycle          [common governed-document concerns]
|- title                  [required]
|- functionalObligation    [required]
|- functionalClause 1..*
```

```

|   | - normativeStatement          [required]
|   |   - spoReference 1..*        [structured governed references]
|   |     | - subject
|   |     | - predicate
|   |     | - object
| - parentDecision -> Decision 1
| - specializedRequirement -> 0..*

```

No FR-local Intent is required in the core. The MR + Decision context should explain why the behavior exists; the FR explains what operationally must happen.

Acceptance/verification evidence and realization artifacts remain downstream relations, not intrinsic FR fields.

4. Operationality rule

An FR must do more than name a desirable state or capability. It must expose enough operational semantics that an implementer/reviewer can understand what must occur.

Strong forms include:

- event/condition -> required transition;
- input/request -> required processing responsibility -> observable result;
- lifecycle event -> state change and failure behavior;
- interaction request -> governed output/correlation behavior;
- concurrency condition -> externally meaningful atomicity/idempotency invariant.

Weak forms such as “provide inventory availability” or “keep authorization updated” are insufficient when they hide responsibility boundaries, lifecycle, concurrency, correlation or failure semantics that materially determine implementation and verification.

5. Decision coverage test

Before accepting an FR, ask:

1. Which parent Decision makes this exact operational responsibility part of the project?
2. Does the Decision determine an internal vs external responsibility boundary, strategy, policy, interface commitment or other material narrowing?
3. If no genuine alternative can be identified, has the branch documented a valid necessity/default Decision rather than silently skipping the layer?
4. Would changing the parent Decision require replacing or materially rewriting this FR? If yes, parentage is semantically strong.
5. Is the proposed parent merely a tautology invented to hold the FR? If yes, strengthen the Decision commitment or use the necessity/default rule transparently.

6. Normative statement plus SPO references

The **normative prose is semantically primary**. SPO makes the participating governed references explicit and reusable; it does not carry all conditional, concurrency, lifecycle or failure semantics by itself.

Readable pattern:

```

[Condition,] <Subject> MUST <Predicate> <Object>
[qualifier / observable outcome / failure behavior].

```

Example:

When a ReservationRequest identifies the active order evaluation and full line set, InventoryService MUST create an ACTIVE Reservation only if the complete set can be allocated atomically against current stock state; a concurrent request MUST NOT reserve quantity already frozen by that Reservation.

Associated SPO references may include:

```
InventoryService -- creates --> Reservation
InventoryService -- updates --> InventoryBalance
InventoryService -- produces --> ReservationResult
```

The structured triples aid identity reuse and tooling, while the normative statement owns atomicity, correlation and failure semantics.

7. FunctionalPredicate metacontract

A governed functional predicate has at least:

- stable identity;
- readable label;
- normative meaning;
- functional area;
- compatible subject/object kinds (to be bound formally after the Base Analysis taxonomy is closed).

The DDTA core does not freeze an exhaustive verb list. Example extension areas include storage, communication, lifecycle, transformation, inventory, cryptography, signature and identity/access.

ThreatForge may later offer predicate pickers filtered by subject/object kinds, but tooling cannot silently invent project semantics.

8. Cross-MR service consumption

MRs may be complementary in an end-to-end service. An FR can consume a capability owned by another MR branch without gaining a second owner.

For the documentation-layer closure, the **semantics** are fixed:

consuming another branch's governed capability does not transfer FR ownership or create a second parent.

The exact canonical target of the non-hierarchical consumption relation is deliberately deferred to the Base Analysis phase. It may resolve to a governed capability/service/BAE/interface relation rather than always to one provider FR.

This avoids prematurely freezing FR -> FR as the only service-composition representation.

9. Parameter classification rule

No generic functionalParameters field belongs to the FR core.

A changing value is classified by semantics:

- image vs frame-set vs short video -> Decision/strategy;
- response-time target -> quality/performance requirement candidate;
- retention duration -> governance/privacy/policy candidate;
- API-imposed limit/field -> InterfaceContract candidate;
- buffer size or implementation tuning -> realization/configuration.

An FR references the applicable governed rule where necessary instead of copying mutable values by default.

10. Closed FR invariants for the documentation baseline

1. **Exactly one parent Decision.**
2. **Derived MR ownership** - MR comes from the parent Decision branch.
3. **Operational obligation** - FR states behavior/service/transition/result, not rationale or architecture preference.
4. **Independent assessability** - satisfaction can be meaningfully evaluated.
5. **Coherent unit** - one independently meaningful capability/behavior per FR.
6. **Decision non-repetition** - FR makes the commitment operational rather than restating it.
7. **Implementation independence** - no incidental framework/source/class/algorithm detail unless contractual.
8. **Normative prose primary** - SPO supports structured references but does not replace full behavior semantics.
9. **SPO consistency** - structured references must not contradict normative prose.
10. **Governed-reference reuse** - existing subject/object identities are referenced rather than redefined.
11. **No Decision children** - later choices remain MR-level Decisions, not descendants of FR.
12. **Specialization downward** - domain-specific constraints enrich/constrain FR through Specialized Requirements.
13. **Service-consumption without co-ownership** - consuming another MR's service/capability does not transfer ownership.
14. **Realization separation** - implementation artifacts realize FRs but are not FR semantics.
15. **Verification separation** - evidence supports satisfaction but is not the FR or its realization.
16. **Method neutrality** - authoring does not require STRIDE/STRIDE-AI terminology.
17. **Tool non-authority** - assistance may resolve/suggest/validate but cannot silently create governed semantics.

11. Specialized Requirement boundary

Conceptually:

FunctionalRequirement

```
|
+-- SecurityRequirement *
+-- GovernanceRequirement *    [future/generalization]
+-- PrivacyRequirement *      [future/generalization]
`-- Performance/Quality ?     [future/generalization]
```

For thesis scope, only SecurityRequirement is required for the next closed end-to-end cycle. Its derivation from accepted Findings and relation to effective obligation will be formalized after Base Analysis and the overlay/plugin contract.

12. Reopen rule

This documentation-layer baseline is reopenable only by a concrete counterexample, especially if:

- SecurityRequirement cannot be attached/constrained without corrupting FR semantics;
- Base Analysis cannot be derived/integrated from the documented system facts without changing MR/Decision/FR authoring;
- a methodology overlay requires information that cannot be expressed without violating the current layer boundaries.